

Model Context Protocol (MCP) Handbook

Tool Standard for Database, Browser & Cloud Integration

Executive Summary: This document provides actionable production implementations engineered by Pinnacle AI Solutions. Use this blueprint to architect zero-downtime, autonomous AI agent pipelines that streamline operational scale and client acquisition.

1. The Universal Standard for Agent Tool Execution

The Model Context Protocol (MCP) standardizes how AI models interface with external databases, browsers, developer tooling, and API gateways. By decoupling model architecture from tool definitions, teams can deploy interoperable server clusters across Claude, Codex, and Antigravity.

Pinnacle AI Solutions leverages a blended architecture: native eager tools for fundamental operations alongside on-demand lazy-loaded servers for specialized infrastructure.

- mcp-server-neon: Zero-latency serverless Postgres query execution, branching, and auto-tuning.
- chrome-devtools & Playwright: Programmatic headless DOM scraping and live web verification.
- Supabase & Cloudflare: Authentication, storage buckets, and serverless edge functions.

2. Authoring Production MCP Connectors

To connect private enterprise APIs, build lightweight TypeScript or Python stdio servers following the JSON-RPC 2.0 schema. Always implement timeout guards and graceful error payloads.

- Declare strict JSON schema validation for all parameters.
- Prevent token exhaustion by truncating large query outputs to 40KB limits.
- Enforce security sandboxes to guard API secrets and database connection strings.

```
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";

const server = new Server({
  name: "pinnacle-auth-bridge",
  version: "1.0.0"
}, { capabilities: { tools: {} } });

server.setRequestHandler(ListToolsRequestSchema, async () => ({
  tools: [{
    name: "verify_subscriber",
    description: "Verifies Pinnacle AI newsletter subscription in Neon DB",
    inputSchema: { type: "object", properties: { email: { type: "string" } }, required: ["email"] }
  }]
}));
```